

Doremi Python vol 2

도레미 파이썬 vol 2

기초 자료형 II

목차 기초 자료형 II

1. 문자열/리스트 활용
2. Tuple
3. Dictionary
4. for 문과 Dictionary

1. 문자열/리스트 활용



list.pop(i)

인덱스 i의 원소를 제거 후, 그 원소를 반환
(괄호를 비우면 마지막 원소를 제거 및 반환)

```
my_list = [2, 3, 5, 7, 11]

print(my_list)
print(my_list.pop(0))
print(my_list.pop())
print(my_list)
```

실행 결과

```
[2, 3, 5, 7, 11]
2
11
[3, 5, 7]
```



seq.count(d)

시퀀스 내부의 자료d의 개수를 반환

```
my_seq = [2, 2, 2, 4, 4]
print(my_seq.count(2))

my_str = "elice and alice"
print(my_str.count("e"))
```

실행 결과

3

3



str.split(c)

c를 기준으로 문자열을 쪼개서 리스트 형태로 반환
(괄호를 비울 시 공백을 기준으로 처리)

```
my_str = "1 2 3 4 5"  
print(my_str.split())  
  
element = "Na,Mg,Al,Si"  
print(element.split(","))
```

실행 결과

```
["1", "2", "3", "4", "5"]  
["Na", "Mg", "Al", "Si"]
```



str.join(list)

str을 기준으로 리스트를 합쳐서 문자열을 반환

```
my_list = ["e", "l", "i", "c",  
"e"]
```

```
print("".join(my_list))
```

```
station = ["서울", "대전",  
"대구"]
```

```
print("→".join(station))
```

실행 결과

elice

서울→대전→대구

2. Tuple



여러 자료를 담는 자료형이 필요하다면?

대부분 여러 자료를 담기 위해서 리스트 자료형을 사용
하지만 리스트 자료형은 원소가 바뀔 위험이 있음

```
my_list = ["e", "l", "i", "c",  
"e"]
```

```
print(my_list)
```

```
my_list[3] = "ERROR"
```

```
print(my_list)
```

실행 결과

```
["e", "l", "i", "c", "e"]
```

```
["e", "l", "i", "ERROR", "e"]
```



값을 바꿀 수 없으면서 여러 자료를 담고 싶을 때는
튜플(Tuple) 자료형을 사용!



튜플 (Tuple)

여러 자료를 함께 담을 수 있는 자료형
소괄호()를 이용하여 표현

```
tuple_zero = ()  
tuple_one = (1,)   
tuple_sample = (1, 2, 3, 4, 5)  
  
print(tuple_sample)  
print(type(tuple_sample))
```

실행 결과

```
(1, 2, 3, 4, 5)  
<class 'tuple'>
```



튜플의 특징 (1)

튜플도 Index를 이용한 인덱싱, 슬라이싱 가능

```
my_tuple = ("A", "C", "E")
```

```
print(my_tuple[1])
```

```
print(my_tuple[:2])
```

실행 결과

C

("A", "C")



튜플의 특징 (2)

in 연산자로 Tuple 안에 원소 확인 가능
len 연산자로 Tuple의 길이 확인 가능

```
my_tuple = ("A", "C", "E")  
  
print("A" in my_tuple)  
  
print(len(my_tuple))
```

실행 결과

```
True  
3
```



튜플의 특징 (3)

+ 연산자로 Tuple과 Tuple을 연결할 수 있음

* 연산자와 숫자를 이용하면 Tuple을 반복할 수 있음

```
tuple_1 = ("G", "O")  
tuple_2 = ("O", "D")  
  
print(tuple_1 + tuple_2)  
  
print(tuple_1 * 3)
```

실행 결과

```
("G", "O", "O", "D")  
("G", "O", "G", "O", "G", "O")
```



튜플의 특징 (4)

튜플은 새로운 자료를 추가하거나 기존의 자료 삭제, 변경이 불가능
한 번 만들어지면 고정되는 자료

```
my_tuple = ("e", "l", "i", "c",  
"e")
```

```
print(my_tuple.append("!"))
```

```
print(my_tuple.remove("e"))
```

```
my_tuple[1] = "ERROR"
```

실행 결과

```
Traceback (most recent call last):  
  File "main.py", line 3, in <module>  
    print(my_tuple.append("!"))  
AttributeError: 'tuple' object has no  
attribute 'append'
```

3. Dictionary



Dictionary?

Dictionary → 사전

사전에서 “단어”와 “뜻”이 하나의 짝꿍인 것처럼 짝꿍이 있는 자료형

dictionary

noun • **UK**  /'dɪkʃən·ər-i/ **US**  /'dɪkʃə·ner-i/ **PLURAL** dictionaries

★ **A1** a book that contains a list of words in alphabetical order with their meanings explained and sometimes written in another language

[사전](#)

Use your dictionaries to look up any words you don't understand.

성

이름

이메일

비밀번호

비밀번호 확인



Dictionary (딕셔너리)

중괄호와 콜론(:)을 이용하여 표현

```
dictionary_zero = {}  
person = {"name" : "Rabbit", "age" : 22}  
  
print(person)
```

실행 결과

```
{"name" : "Rabbit", "age" : 22}
```



짝궁이 있는 자료형이므로
Key를 알면 Value를 알 수 있음

```
dictionary_zero = {}  
person = {"name" : "Rabbit", "age" : 22}  
  
print(person)
```

실행 결과

```
{"name" : "Rabbit", "age" : 22}
```

열쇠처럼 딕셔너리에서 자료를 꺼낼 수 있는 도구

```
dictionary_zero = {}  
person = {"name" : "Rabbit", "age" : 22}  
  
print(person)
```

실행 결과

```
{"name" : "Rabbit", "age" : 22}
```

딕셔너리에서 Key를 이용하여 꺼낸 자료

```
dictionary_zero = {}  
person = {"name" : "Rabbit", "age" : 22}  
  
print(person)
```

실행 결과

```
{"name" : "Rabbit", "age" : 22}
```




Dictionary[Key]

딕셔너리에서 Key를 이용하여 자료(value) 꺼내기

```
person = {"name" : "Rabbit", "age" : 22}

print(person["name"])
print(person["age"])
```

실행 결과

Rabbit

22



딕셔너리에서 자료 추가하기

```
person = {  
    "name" : "Rabbit",  
    "age" : 22  
}  
  
person["mail"] =  
    "rabbit@elice.com"  
  
print(person)
```

실행 결과

```
{"name": "Rabbit",  
"age": 22,  
"mail": "rabbit@elice.com"}
```



Dictionary[Key]

del을 이용하면 Dictionary의 원소(Key와 Value) 삭제 가능

```
person = {  
    "name" : "Rabbit",  
    "age" : 22,  
    "mail" : "rabbit@elice.com"  
}  
  
del person["age"]  
  
print(person)
```

실행 결과

```
{"name": "Rabbit",  
"mail": "rabbit@elice.com"}
```

keys() 를 이용하면 Dictionary 의 key 들만 추출 가능

```
person = {  
    "name" : "Rabbit",  
    "age" : 22,  
    "mail" : "rabbit@elice.com"  
}  
  
print(person.keys())
```

실행 결과

```
dict_keys(['name', 'age', 'mail'])
```

values() 를 이용하면 Dictionary 의 value 들만 추출 가능

```
person = {  
    "name" : "Rabbit",  
    "age" : 22,  
    "mail" : "rabbit@elice.com"  
}  
  
print(person.values())
```

실행 결과

```
dict_values(['Rabbit', 22,  
'rabbit@elice.com'])
```


items()를 이용하면 Dictionary의 Key와 Value의 쌍 추출 가능

```
person = {  
    "name" : "Rabbit",  
    "age" : 22,  
    "mail" : "rabbit@elice.com"  
}  
  
print(person.items())
```

실행 결과

```
dict_items([('name', 'Rabbit'), ('age', 22),  
            ('mail', 'rabbit@elice.com')])
```




Dictionary의 특징

Key는 변할 수 없는 자료형이어야 함
→ 리스트는 사용할 수 없고 튜플은 사용할 수 있음

```
dictionary_example = {[1, 2, 3] : "Hello"}  
  
print(dictionary_example)
```

실행 결과

```
dict_items([('name', 'Traceback (most recent call last):  
File "main.py", line 1, in <module>  
dictionary_example = {[1, 2, 3] : "Hello"}  
  
TypeError: unhashable type: 'list'  
Rabbit'), ('age', 22),  
(('mail', 'rabbit@elice.com'))])
```

```
dictionary_example = {(1, 2, 3) : "Hello"}  
  
print(dictionary_example)
```

실행 결과

```
{(1, 2, 3) : "Hello"}
```

4. for 문과 Dictionary



딕셔너리 자료형화 반복문

- 딕셔너리 자료형은 Key와 Value의 쌍으로 이루어진 자료형
- 딕셔너리 자료형에서 제공하는 다양한 기능과 반복문을 이용하면 효과적인 분석 가능

```
dictionary_zero = {}  
person = {"name" : "Rabbit", "age" : 22}  
  
print(person)
```

실행 결과

```
{"name" : "Rabbit", "age" : 22}
```



for 문의 변수는 여러 개를 사용할 수 있음

```
week = [(1, "월"), (2, "화"),  
        (3, "수"), (4, "목"), (5,  
        "금"), (6, "토"), (7, "일")]
```

```
for num, date in week :  
    print(num)  
    print(date)
```

실행 결과

```
1  
월  
2  
화  
...  
7  
일
```


- 여러 리스트, 튜플, 문자열 등의 데이터를 병렬로 묶어 튜플로 반환하는 명령어
- 동일한 인덱스에 있는 요소들을 하나의 튜플로 묶음

```
list1 = ["A", "B", "C", "D"]  
list2 = [11, 13, 17, 19]  
  
print(zip(list1, list2))  
print(list(zip(list1, list2)))
```

실행 결과

```
<zip object at 0x7f47defdf188>  
[('A', 11), ('B', 13), ('C', 17), ('D', 19)]
```

```
list1 = ["A", "B", "C", "D"]  
list2 = [11, 13, 17, 19]  
  
print(dict(zip(list1, list2)))
```

실행 결과

```
{'A': 11, 'B': 13, 'C': 17, 'D': 19}
```




Key, Value 구분하여 출력하기 (1)

반복문과 zip, Dictionary의 특징을 이용하면
Key와 Value를 구분하여 출력할 수 있음

```
my_dict = {"a": 11, "b": 13, "c": 17}

for key, value in zip(my_dict.keys(), my_dict.values()) :
    print("key: ", key, ", value: ", value)
```

실행 결과

```
key: a, value: 11
key: b, value: 13
key: c, value: 17
```



Key, Value 구분하여 출력하기 (2)

반복문과 Dictionary의 특징을 이용하면
Key와 Value를 구분하여 출력할 수 있음

```
my_dict = {"a": 11, "b": 13, "c": 17}

for key, value in my_dict.items():
    print("key: ", key, ", value: ", value)
```

실행 결과

```
key: a, value: 11
key: b, value: 13
key: c, value: 17
```